

Sistemas Operacionais 2

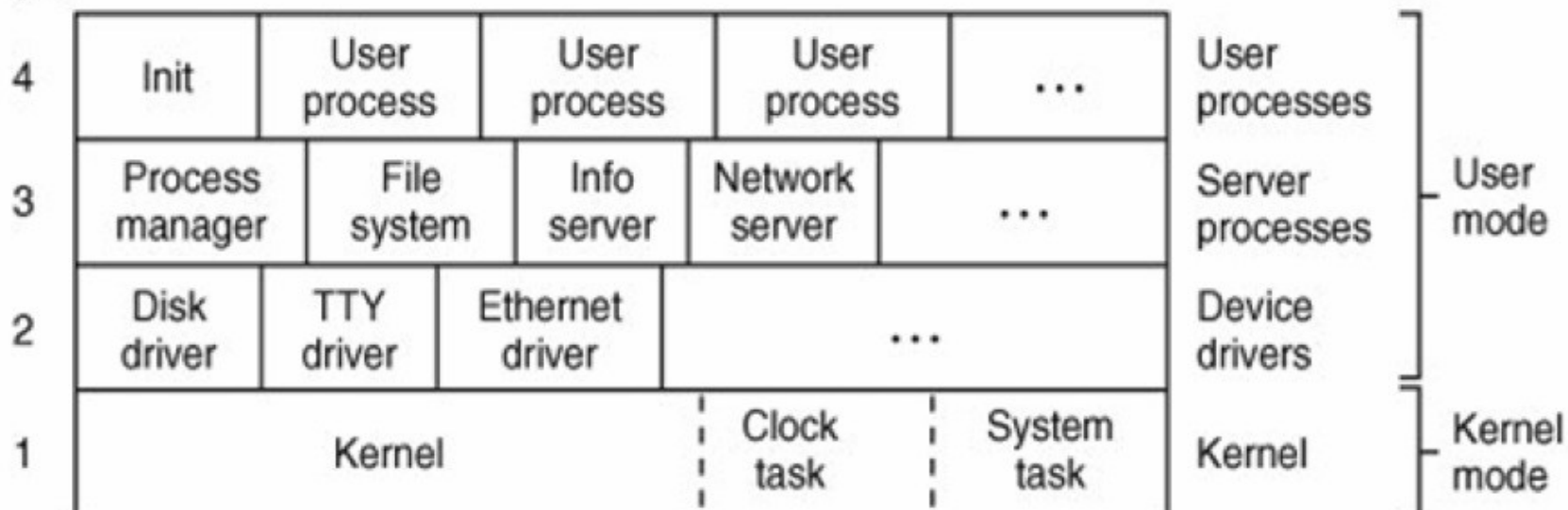
INTRODUÇÃO AO MINIX

Professor.: Dalton Matsuo Tavares

Nota: Alguns slides e/ou figuras nesta apresentação foram adaptadas de slides de TANENBAUM e WOODHULL ©2006. Cortesia de Dalton Matsuo Tavares.

Estrutura Interna do MINIX

Layer



- Estruturado em 4 camadas:

- Camada 1: o kernel do MINIX é dividido em módulos, os quais se comunicam por meio de uma única primitiva de comunicação.
- Camadas 2 e 3 possui privilégios para realizar chamadas de kernel.
- Camada 4: processos não possuem privilégios especiais.

- Escalona processos e gerencia as transições entre os estados pronto, executando e bloqueado.
- Lida com todas as mensagens entre processos
- Acesso a portas de I/O e interrupções
- Tarefas de relógio
- Tarefas de sistema

Camada 2: Controladores de Dispositivo

- Projeto para controladores de dispositivo
- Acesso a portas de I/O
- Um controlador é necessário para cada tipo de dispositivo
- A camada 2 possui a maioria dos privilégios para fazer chamadas de sistema.

Camada 3: Processos de Servidor

- Camada 3 é pretendida para servidores oferecendo serviços úteis
- Gerenciador de processo (*Process Manager – PM*) e sistema de arquivo (*File System – FS*) são essenciais.
- A camada 3 possui alguns privilégios para fazer chamadas de kernel.

Camada 4 e Chamadas de Sistema POSIX

- Chamadas de sistema POSIX x chamadas de kernel.
- A camada 4 é para processos de usuário.
- Processo init.

```

load averages: 3.25, 2.56, 0.86
34 processes: 4 running, 30 sleeping
Mem: 251360K Free, 251000K Contiguous Free
CPU states: 100.00% user, 0.00% system, 0.00% kernel, 0.00% idle

  PID USERNAME PRI NICE  SIZE  STATE  TIME   CPU COMMAND
  103 root      11  12    8K   RUN   0:42  34.90% mytest
  104 root      10   9    8K   RUN   1:02  33.56% mytest
  105 root       9   7    8K   RUN   2:11  31.54% mytest
[ -3] root       0      76K   0:00  0.00% clock
[ -2] root       0      76K   0:01  0.00% system
[ -1] root       0      76K   0:00  0.00% kernel
    0 root       3 -10   120K  0:00  0.00% pm
    4 root       5  -8  4956K  0:00  0.00% fs
    5 root       6 -10    40K  0:00  0.00% rs
    8 root       2 -13   304K  0:00  0.00% mem
    9 root       2 -13    76K  0:00  0.00% log
    7 root       3 -16    84K  0:00  0.00% tty
    6 root       3 -10    24K  0:00  0.00% ds
    1 root       0   0    16K  0:00  0.00% init
   12 root       6 -10    44K  0:00  0.00% pci
   14 root       3 -10    20K  0:00  0.00% floppy
   17 root       7 -10    80K  0:00  0.00% at_wini
  
```

- Três primitivas para enviar e receber mensagens:
 - `send(dest, &message);`
 - `receive(source, &message);`
 - `sendrec(src_dst, &message);`
- Oferecido por uma biblioteca C. Mecanismo interno ao kernel.
- Restrição de passagem de mensagem
 - Princípio de *rendezvous*

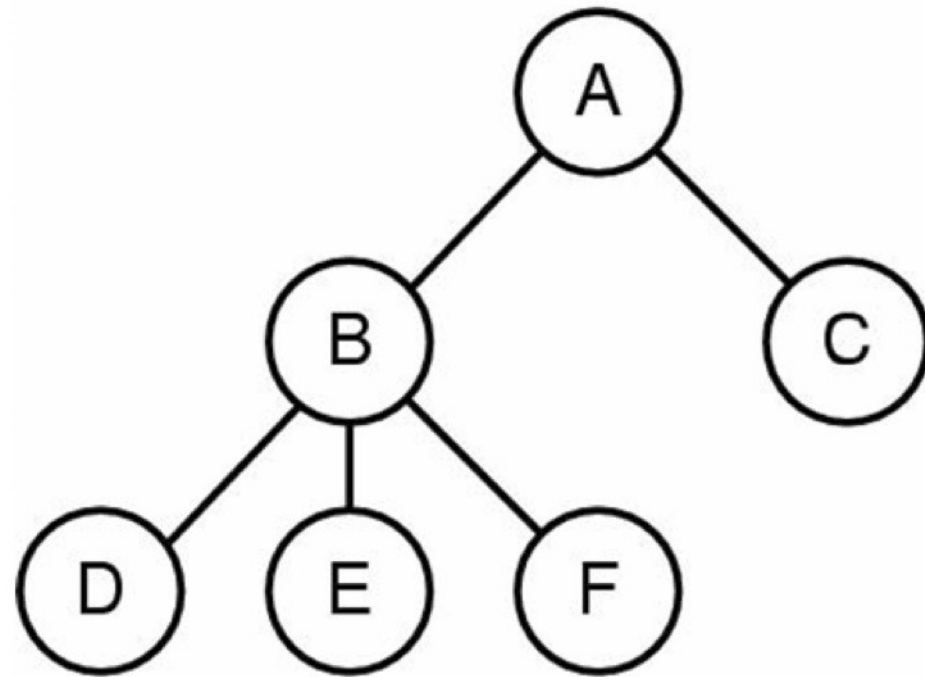
Inicialização do MINIX

- O Minix possui um programa de *boot*.
- A imagem de *boot* define os processos a serem carregados na memória.
- O *init* é o último processo a ser executado na sequência de boot.

Component	Description	Loaded by
kernel	Kernel + clock and system tasks	(in boot image)
pm	Process manager	(in boot image)
fs	File system	(in boot image)
rs	(Re)starts servers and drivers	(in boot image)
memory	RAM disk driver	(in boot image)
log	Buffers log output	(in boot image)
tty	Console and keyboard driver	(in boot image)
driver	Disk (at, bios, or floppy) driver	(in boot image)
init	parent of all user processes	(in boot image)
floppy	Floppy driver (if booted from hard disk)	/etc/rc
is	Information server (for debug dumps)	/etc/rc
cmos	Reads CMOS clock to set time	/etc/rc
random	Random number generator	/etc/rc
printer	Printer driver	/etc/rc

Árvore de Processo no MINIX

- Gerenciador de processo possui o pid 0.
- O init é o primeiro processo de usuário e “avô” de todos os processos de usuário.
- Kernel, tarefas de sistema e relógio não estão nesta árvore de processo.



- `/usr/src`
- `/usr/src/include` onde estão localizados os arquivos de cabeçalho C.
- `/usr/include`, `#include <..>`
- Cada diretório possui seu próprio Makefile
- Execute o make dentro de `/usr/src/tools`